

Advanced Learning Algorithms

Course 2 — detailed study notes (Weeks 1–4)

Neural networks · training · ML advice · decision trees

1. Week 1 — Neural Networks (inference)

1.1 From logistic unit to neuron

A single **neuron** is exactly logistic regression: it takes inputs, computes $z = \mathbf{w} \cdot \mathbf{x} + b$, applies an activation g , outputs $a = g(z)$. That output a (the “activation”) is passed to the next layer. Stack many neurons in layers $\Rightarrow$ a neural network that learns its own features.

1.2 Layer notation

Superscript $[l]$ = layer number; subscript j = unit within the layer.

$$a_j^{[l]} = g\left(\mathbf{w}_j^{[l]} \cdot \mathbf{a}^{[l-1]} + b_j^{[l]}\right), \quad \mathbf{a}^{[0]} = \mathbf{x}.$$

Each layer takes the previous layer’s activation vector $\mathbf{a}^{[l-1]}$ as its input. The final layer’s output $\mathbf{a}^{[L]}$ is the prediction. For binary classification the last unit is a sigmoid, so $a^{[L]}$ is a probability.

1.3 Forward propagation

“Inference” = running the network forward, layer by layer, to get a prediction:

$$\mathbf{x} \rightarrow \mathbf{a}^{[1]} \rightarrow \mathbf{a}^{[2]} \rightarrow \dots \rightarrow \mathbf{a}^{[L]} = \hat{y}.$$

Layers near the input learn simple features (edges), deeper layers combine them into complex ones (faces) — for image tasks especially.

1.4 TensorFlow / Keras implementation

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(units=25, activation='sigmoid'), # layer 1
    Dense(units=15, activation='sigmoid'), # layer 2
    Dense(units=1, activation='sigmoid')   # output
])
model.compile(loss='binary_crossentropy')
model.fit(X, y, epochs=100)
model.predict(X_new)
```

Data shape: TensorFlow expects 2-D arrays. A single example is a row `[[200, 17]]` (shape 1×2), not `[200,17]`. NumPy `np.array([[...]])` with double brackets.

1.5 Vectorized forward prop

For a whole layer, stack weights into a matrix W and use matrix multiply:

$$\mathbf{Z} = \mathbf{A}_{\text{in}} \mathbf{W} + \mathbf{b}, \quad \mathbf{A}_{\text{out}} = g(\mathbf{Z}).$$

```
def dense(A_in, W, b, g):
    Z = np.matmul(A_in, W) + b
    return g(Z)
```

`np.matmul` (or the `@` operator) is why networks run fast — the whole layer is one matrix product on optimized hardware.

2. Week 2 — Training & Activations

2.1 The 3 training steps (same as logistic regression)

1. **Define the model** $f(\mathbf{x})$ — the architecture.
2. **Specify the loss/cost** — how wrong the prediction is.
3. **Minimize the cost** — gradient descent (via backprop).

```
model.compile(loss=BinaryCrossentropy()) # step 2
model.fit(X, y, epochs=100)             # step 3 (backprop)
```

The loss for binary classification is the same log loss as Course 1 (“binary crossentropy”); for regression it’s mean squared error.

2.2 Activation functions

Name	Formula	Use
Linear	$g(z) = z$	regression output (y any value)
Sigmoid	$g(z) = \frac{1}{1+e^{-z}}$	binary output ($y \in \{0, 1\}$)
ReLU	$g(z) = \max(0, z)$	default for hidden layers

Why ReLU for hidden layers

ReLU is cheap and doesn’t saturate for large z , so gradients stay healthy and learning is faster. Sigmoid flattens at both ends (gradient $\rightarrow 0$), which slows gradient descent.

Why we need a non-linear activation at all: if every layer were linear, the whole network would collapse to a single linear function — no more expressive than plain linear regression. The non-linearity is what lets networks model complex relationships.

2.3 Multiclass classification — Softmax

For N classes, softmax generalizes the sigmoid. Given $z_k = \mathbf{w}_k \cdot \mathbf{x} + b_k$ for each class k :

$$a_k = \frac{e^{z_k}}{\sum_{j=1}^N e^{z_j}} = P(y=k \mid \mathbf{x}), \quad \sum_{k=1}^N a_k = 1.$$

Loss (sparse categorical crossentropy): if the true class is k , loss $= -\log a_k$.

Numerically stable softmax

Computing e^z then dividing loses precision. Instead, let the output layer be *linear* and fold the softmax into the loss:

```
Dense(units=10, activation='linear') # output raw logits
model.compile(
    loss=SparseCategoricalCrossentropy(from_logits=True))
# then apply softmax to model outputs at predict time
```

Multiclass vs. multi-label: multiclass = one label out of many (digit is 0–9). Multi-label = several yes/no tags at once (image contains car? bus? pedestrian?) — use several sigmoid outputs, not softmax.

2.4 Adam optimizer

An upgrade over plain gradient descent: it keeps a *separate, adaptive* learning rate per parameter and adjusts them on the fly (speeds up when progress is steady, slows down when it overshoots).

```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
              loss=...)
```

Almost always faster than vanilla gradient descent; a safe default.

2.5 Backpropagation (optional)

Backprop computes $\partial J / \partial w$ for every weight efficiently using the **chain rule** on a **computation graph**. It works right-to-left: compute the output error, then propagate it backward through each layer, reusing intermediate results so the cost is roughly the same as one forward pass. TensorFlow does this automatically inside `fit`.

3. Week 3 — Advice for Applying ML

3.1 Evaluating a model

Split data: **training** / **cross-validation (dev)** / **test**, e.g. 60/20/20.

- Fit parameters w, b on the **training** set.
- Choose the model / degree / λ using the **CV** set.
- Report final performance *once* on the **test** set.

Never pick your model using the test set — if you do, the test error is optimistic and no longer an honest estimate of real-world performance.

3.2 Diagnosing bias vs. variance

Compare training error J_{train} and CV error J_{cv} :

	High bias (underfit)	High variance (overfit)
J_{train}	high	low
J_{cv}	high, $\approx J_{train}$	high, $\gg J_{train}$

Baseline: compare against a reference (human performance, or a known benchmark). Bias = gap between baseline and J_{train} ; variance = gap between J_{train} and J_{cv} .

Regularization λ and bias/variance

Large $\lambda \Rightarrow$ strong penalty $\Rightarrow$ underfit (high bias). Small $\lambda \Rightarrow$ overfit (high variance). Sweep λ over a range and pick the value that minimizes J_{cv} .

3.3 Learning curves

Plot J_{train} and J_{cv} vs. training-set size m .

- **High bias:** both curves flatten at a high error and are close together. *More data won't help* — fix the model instead.

- **High variance:** large gap between the two curves. *More data helps* — it closes the gap.

What to try next

Fixes high variance

get more training examples
smaller set of features
increase λ

Fixes high bias

add features
add polynomial features
decrease λ

Neural nets & bias/variance: a bigger network almost always lowers bias. Recipe: (1) if J_{train} high $\rightarrow$ bigger network; (2) if J_{cv} high $\rightarrow$ more data or regularization; repeat. Large regularized networks rarely hurt except for compute cost.

3.4 ML development process

1. Choose architecture $\rightarrow$ train $\rightarrow$ diagnose (bias/variance + error analysis) $\rightarrow$ repeat.
2. **Error analysis:** manually inspect a sample of misclassified CV examples, group them by cause, and spend effort where the biggest category is.
3. **Adding data:** data augmentation (distort/rotate images, add background noise to audio) creates realistic new examples cheaply; data synthesis generates new ones from scratch.
4. **Transfer learning:** take a network pretrained on a large dataset, keep its early layers, and fine-tune only the output layer(s) on your small dataset. Early layers already learned generic features.

3.5 Skewed datasets — precision / recall

When one class is rare, accuracy is misleading (predicting “always negative” scores 99.5%). Use a confusion matrix (TP, FP, FN, TN):

$$P = \frac{TP}{TP + FP} \text{ (precision), } R = \frac{TP}{TP + FN} \text{ (recall),}$$

$$F_1 = 2 \frac{P \cdot R}{P + R} \text{ (harmonic mean).}$$

- Raise the classification threshold $\Rightarrow$ higher precision, lower recall.
- Lower it $\Rightarrow$ higher recall, lower precision.
- F_1 balances the two into one number; pick the threshold/model with the best F_1 .

4. Week 4 — Decision Trees

4.1 The model

A tree of yes/no questions on features. Each internal node splits on a feature; each leaf assigns a class (or value). Prediction = walk from root to leaf following the answers.

4.2 Measuring purity — entropy

A node’s impurity, where p_1 = fraction of positive examples at that node:

$$H(p_1) = -p_1 \log_2(p_1) - (1 - p_1) \log_2(1 - p_1).$$

$H = 0$ when the node is pure ($p_1 = 0$ or 1); $H = 1$ at maximum mix ($p_1 = 0.5$). Convention: $0 \log 0 = 0$.

4.3 Choosing a split — information gain

Pick the split that most reduces entropy. Information gain = parent entropy minus the weighted average entropy of the children:

$$\text{IG} = H(p_1^{\text{root}}) - \left(w^{\text{left}} H(p_1^{\text{left}}) + w^{\text{right}} H(p_1^{\text{right}}) \right),$$

where $w^{\text{left}}, w^{\text{right}}$ are the fractions of examples going each way. Choose the feature with the highest IG at every node.

Building the tree (recursion)

Start at the root with all data; pick the max-IG split; partition; recurse on each child. **Stop** when: a node is pure, max depth is reached, IG is below a threshold, or the node has too few examples (these limits control overfitting).

4.4 Handling feature types

- **Categorical (> 2 values): one-hot encode** into separate 0/1 features.
- **Continuous:** try thresholds (e.g. $x < 8.5$), pick the threshold giving highest IG.
- **Regression trees:** leaf predicts the *mean* of its examples; split to reduce **variance** instead of entropy.

4.5 Tree ensembles

A single tree is high-variance (sensitive to small data changes). Combining many trees fixes this.

- **Sampling with replacement** (bootstrap): draw m examples with repeats to build each tree on a slightly different dataset.
- **Random forest:** many bootstrapped trees; at each split consider only a random subset of $k \approx \sqrt{n}$ features. Predict by majority vote. Decorrelating the trees is what makes the ensemble strong.
- **XGBoost:** boosted trees — each new tree focuses on the examples previous trees got wrong. Fast, regularized, a top choice for tabular data.

4.6 Trees vs. neural networks

Decision trees / ensembles

great on *structured/tabular* data

fast to train

ensembles usually beat single trees

Neural networks

great on *unstructured* (image, audio, text)

slower, but works with transfer learning

can chain multiple networks

Two big ideas of Course 2: (1) a neural network is layered logistic units trained by the same define→loss→minimize recipe, with ReLU hidden layers and softmax for multiclass; (2) most of the real work is *diagnosis* — use bias/variance, learning curves, and error analysis to decide what to fix next, rather than guessing.